SkillClaw: Let Skills Evolve Collectively with Agentic Evolver

Collective Skill Evolution for Multi-User LLM Agent Ecosystems

arXiv:2604.08377

Agent Skills Remain Static After Deployment — Users Rediscover Solutions Independently

Static Skill Paradigm

Skills manually installed from a centralized hub and never updated post-deployment.

Failures and recovery patterns are trapped inside individual sessions.

Memory-based methods store trajectories but remain tied to specific instances and are hard to generalize.

Skill-based methods compress experience into instructions but treat the library as a static resource.

Desired: Evolving Skill Ecosystem

Runtime interaction trajectories continuously feed back into skill improvement.

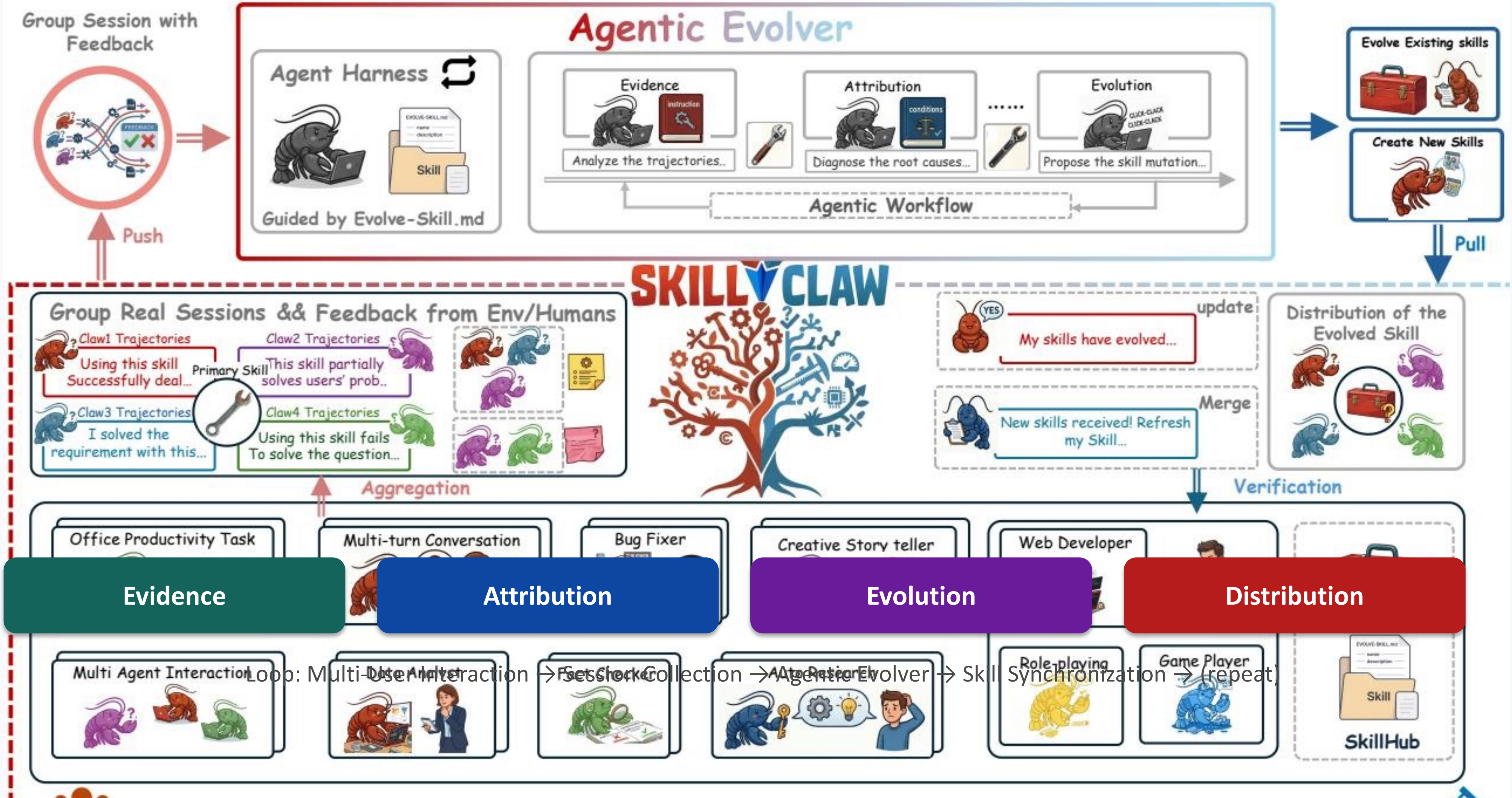
Cross-user patterns aggregate into a shared evidence base.

Skills refine or mutate based on observed success and failure modes.

The system accumulates knowledge rather than forcing each user to rediscover solutions.

Key insight: Similar tasks recur across users and time, yet the system does not improve — each user is forced to rediscover independently.

Closed-Loop Pipeline: From Multi-User Interaction to Shared Skill Evolution



Evidence | **Attribution** | **Evolution** | **Distribution**

Loop: Multi-User Interaction → Session Collection → Agentic Evolver → Skill Synchronization → (repeat)

Agentic Evolver: Evidence → Attribution → Evolution in Three Stages

1. Evidence

Analyze trajectories to identify behavioral patterns

Aggregate structured session trajectories across users

Preserve full action–feedback causal chains

Group trajectories by referenced skills

Surface success patterns and failure modes

2. Attribution

Diagnose root causes of failures and successes

Open-ended reasoning over interaction evidence

Not predefined update rules

Identify why a skill breaks under specific contexts

Distinguish generalizable issues from idiosyncratic fixes

3. Evolution

Propose skill mutations and verify before merging

Refine existing skills or create new ones

Mutations are verified against held-out trajectories

Conservative acceptance prevents regressions

Merged skills synchronize back to all agents

Emphasis: The evolver is an autonomous agent performing open-ended reasoning — not a set of predefined update rules.

Cross-User Trajectory Aggregation Exposes Skill Behavioral Boundaries

Why a single user is not enough

Different users exercise the same skill under diverse contexts, producing complementary views.

These views reveal both conditions where the skill works and where it breaks.

A single user rarely generates enough signal to separate a generalizable improvement from an idiosyncratic fix.

Aggregation groups trajectories by referenced skills, forming a shared evidence base.

Compatible Claw-Style Systems

OpenClaw • CoPaw • IronClaw • PicoClaw • ZeroClaw • NanoClaw • NemoClaw

Evaluation on WildClawBench using Qwen3-Max as the backbone model demonstrates substantial improvements across tasks.

Insight: Cross-user aggregation provides complementary behavioral signals that a single user cannot generate alone.

Case Study: Multimodal Creative Pipeline Skill — 6 Days of Evolution Attempts

Day	Candidate Skill	Skill Function	Outcome
1	validate-tmp-workspace-inputs	Check /tmp_workspace inputs & environment setup	**Accepted**
2	multimodal-input-validation-and-setup	Multimodal input validation & output env init	**Rejected**
3	multimodal-creative-task-pipeline	Multimodal creative pipeline (extract from PDF/video/image)	**Rejected**
4	multimodal-creative-task-pipeline (impr.)	Added image classification, visual generation, structured validation	**Rejected**
5	multimodal-creative-task-pipeline (impr.) + validate-required-input-files	Creative pipeline + per-file fail-fast validation	**Rejected**
6	multimodal-creative-task-pipeline (cand.)	Extended PDF-to-poster / document-to-visual paths	**Rejected**

Result: Only 1 of 6 candidates accepted — conservative verification prevents regressions.

Case Study: Git Push Auth-Fallback Skill — Iterative Refinement Over 6 Days

Day	Candidate Skill	Skill Function	Outcome
1	git-push-with-auth-fallback	Safe fallback instead of blocking on push failure	Accepted
2	git-push-with-auth-fallback	Unified patch/bundle filenames, reduced inconsistency	Accepted
3	git-push-with-auth-fallback + git-clone-to-directory	Auth-alternative paths & secrets audit; fixed subshell pitfalls	Accepted
4	(none)	Same-pool retest; validator confirmed current pool as best	Accepted
5	git-push-with-auth-fallback	Added 'push hang treated as auth failure'; no improvement	Rejected
6	git-push-with-auth-fallback	Added identity config & filename consistency; no improvement	Rejected

Result: 3 of 6 days yielded accepted updates — successful iterative refinement before reaching a plateau.

Key Takeaways: Three Properties That Distinguish SkillClaw from Existing Systems

1 Collective Evolution

Knowledge from individual interactions contributes to a shared, continuously improving skill ecosystem.

2 Fully Automatic

Skill evolution is driven by runtime interaction without manual curation or explicit user intervention.

3 Agentic Evolution Paradigm

Skill updates are produced through open-ended reasoning rather than predefined update rules.

Implications for Building Continuously Improving Agent Systems

SkillClaw shows that agent capabilities can improve at the ecosystem level — not by retraining models, but by closing the loop between runtime experience and structured skill evolution. This opens a path toward self-improving agent infrastructures that learn from their users.